

Murine antibody humanization via frequentist analysis and simulated annealing

Eneko Acevedo, Íñigo Aliod

Contents

1	Intro	3
1.1	Raw data files	3
1.2	Aminoacid's properties	3
2	Representation and modelization of data	3
2.1	Aminoacids and properties	3
2.2	Chains	4
3	Expressions and Formulas	4
3.1	Entropies	4
3.2	Correlation	4
4	Metropolis	5
4.1	Simulation Workflow: From Batch to Single Run	5
4.1.1	The <code>mega.metropolis</code> Manager	5
4.1.2	The <code>run.metropolis</code> Engine	6
4.2	Simulated Annealing Schedule (β)	6
4.3	Energy Function	6
4.4	Model Parameters	7

1 Intro

The aim of this GitHub repository is to delve deeper into biophysics, using simulated annealing and science data to humanize murine antibodies.

We use a frequentist analysis of the given data in `learn_human.txt` and `learn_mouse.txt` in order to measure entropies and correlations.

With those outputs we intend to fine-tune the simulated annealing algorithm for the humanization.

1.1 Raw data files

The `learn_human.txt` and `learn_mouse.txt` data entries are chains of 298 aminoacids each. Every aminoacid is represented by a character in this string: `-ACDEFGHIKLMNPQRSTVWY`. The `-` is a gap in the chain, as used in the notation convention.

1.2 Aminoacid's properties

Every aminoacid is a certain chemical compound. Thus, it has chemical properties, which vary from aminoacid to aminoacid. The used properties in this project are (in order): porperty-gap, hydrophobic, aromatic, aliphatic, polar, small, minuscule, positively charged, and negatively charged. Every aminacid has assigned at least one property, so for consistency we added the property version of the gap: property-gap, denoted as `-(PROP)`.

2 Representation and modelization of data

2.1 Aminoacids and properties

Every aminoacid will be simulated/coded as a 21-dimensional vector of real numbers between 0 and 1 yuxtaposed to a 9-dimensional vector of real numbers between 1 and 0. That is, some aminoacid in our aminoacid space, $a \in \mathcal{A}$, will be $a = v \times w \in [0, 1]^{21} \times [0, 1]^9 = \mathcal{A} \subset \mathbb{R}^{21} \times \mathbb{R}^9$.

- The first list will be a 21-dimensional vector, whose i -th component corresponds to the frequency of the i -th aminoacid from the aforementioned aminoacid string.
- The second list will be a 9-dimensional vector, whose i -th component corresponds to the frequency of the i -th property from the aforementioned property list.

As you can see, these variables are context-dependent, since there is no reference for the frequency of aminoacids or properties. Therefore, if it is one that represents an actual aminoacid, all frequencies may be 0 but one of them, which should be 1. Same will happen for the property vector. We will call this a *concrete aminoacid*.

Example: Let's encode the aminoacid cysteine (`C`). It is the third in the aminoacid list, thus, the first vector should be $v = (0, 0, 1, 0, \dots, 0)$. Cysteine is hydrophobic and small, then: $w = (0, 1, 0, 0, 0, 1, 0, 0, 0)$, since hydrophobic is in the 2nd position and small in the 6th.

Now changing the context, another usual setting is the so called *Schrödinger's aminoacid*. That will be the average aminoacid, whose elements in both vectors will be *separately* the frequencies of every appearing aminoacid.

Separately means that the sum of the components of the first list is 1 and the sum of the components of the second list will also be 1.^[1] More specifically: $\sum_{i=1}^{21} v^i = 1 \wedge \sum_{i=1}^9 w^i = 1$.^[2]

2.2 Chains

A chain is an array of 298 aminoacids and it represents a whole antibody. Just as before, we will have concrete (all components must be concrete aminoacids) and Schrödinger's chains (the rest).

This is a $298 \times (21 + 9)$ matrix of split row conditioned frequencies. We will name the space of all possible chains the chain space, $\mathcal{C} = \mathcal{A}^{298}$. Thus, $c \in \mathcal{C}$ will have the form $c = (c_1, \dots, c_j, \dots, c_{298})$, where $c_j = v_j \times w_j$ and $v_j = (v_j^1, \dots, v_j^i, \dots, v_j^{21})$, $\forall j \in [1, 298] \subset \mathbb{N}$.

3 Expressions and Formulas

3.1 Entropies

We quantify the variability of the aminoacids in a Schrödinger's chain with entropies, applying its expression to both vectors of every aminoacid in that chain. This means the result will be a 298 element list of pairs of entropies.

The compute them as follows:

$$S_{v,j} = - \sum_{i=1}^{21} v_j^i \log_2 v_j^i, \quad S_{w,j} = - \sum_{i=1}^9 w_j^i \log_2 w_j^i, \quad S_j = (S_{v,j}, S_{w,j}) \quad \forall j \in [1, 298] \subset \mathbb{N}$$

That previous one is the Shannon entropy, but we may need to use some others:

- Linear entropy: $s = \sum_i p_i(1 - p_i)$
- Rényi entropy: $s_\alpha = \frac{1}{1-\alpha} \sum_i p_i^\alpha$, $\alpha \in [0, 1)$
- Tsallis entropy: $s_\alpha = \frac{1}{q-1} (1 - \sum_i p_i^\alpha)$, $\alpha \in [0, 1)$

3.2 Correlation

The measure of correlation between positions in a chain used in this project is mutual information. It is an entropy measure of how much information (in bits) we can deduce of one random variable given another one, and vice versa.

Mutual information is given by:

$$I(X; Y) = \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} P(\{X = x\} \cap \{Y = y\}) \log_2 \frac{P(\{X = x\} \cap \{Y = y\})}{P(\{X = x\})P(\{Y = y\})},$$

¹This is true for every kind of aminoacid, whether it is concrete or Schrödinger's.

²Using superindexes for convenience.

where X and Y are discrete random variables that take values, x and y , in $\mathcal{X}$ and $\mathcal{Y}$. Since we can identify our vectors, v and w , with distribution functions, we are able to apply this method.

Note that $I(X;Y) = I(Y;X)$, since the product and intersection are symmetric.

Also,

$$I(X;X) = \sum_{x \in \mathcal{X}} P(\{X = x\}) \log_2 \frac{P(\{X = x\})}{(P(\{X = x\}))^2} = - \sum_{x \in \mathcal{X}} P(\{X = x\}) \log_2 P(\{X = x\}) = S(X),$$

where $S(X)$ is the entropy of the random variable X .

In practice, we will get a 298×298 symmetric matrix for aminoacids (v -part of the chain) whose diagonal is the entropies of every position. However, for our correlation indeces we would rather have all 1's in the diagonal and the other elements going from 0 to 1. We normalize it as follows:

$$i(X;Y) = \frac{I(X;Y)}{\min\{S(X), S(Y)\}}.$$

When $\min\{H(X), H(Y)\} = 0$, the position is fully conserved and $I(X;Y) = 0$ as well. In this case, the code outputs $i(X;Y) \equiv 0$, by convention.

4 Metropolis

4.1 Simulation Workflow: From Batch to Single Run

Two functions are used for running the simulation: a high-level manager (`mega_metropolis`), and a core simulation engine (`run_metropolis`). This design allows for parallelized execution of simulations on multiple initial murine sequences.

4.1.1 The `mega_metropolis` Manager

`mega_metropolis`'s primary responsibilities are:

1. **Data Loading:** It begins by reading a list of initial murine sequences (which we call seeds) from a specified file and saves them for later use.
2. **Batch Management:** It creates a timestamped main directory for the entire simulation batch (format `results/metropolis/YYYY-MM-DD_HH_MM_SS_.../`). .
3. **Parallel Execution:** It iterates through each seed sequence using an OpenMP parallel loop. For each seed, it creates a dedicated subdirectory (e.g., `seq_1`, `seq_2`).
4. **Replication:** For each seed, it calls the `run_metropolis` function `n_metropolis` times. This allows for multiple independent simulation runs starting from the same seed, providing statistical significance to the results. The output of each run is saved to a separate file (e.g., `run_1.txt`, `run_2.txt`) within the corresponding seed's subdirectory.

4.1.2 The run_metropolis Engine

The `run_metropolis` function performs a single, complete simulated annealing run on one given sequence:

1. **Initialization:** It takes an initial sequence, the human Schrödinger chain, and the pre-calculated matrix of β values.
2. **Simulated Annealing Loop:** It iterates through each row of the β matrix, representing a step in the “cooling” schedule.
3. **Metropolis Sweeps:** For each β^i step, it performs a specified number of Metropolis sweeps (`n_sweeps`). A single sweep involves attempting a mutation at every position in the sequence.
4. **State Acceptance:** We begin by proposing a new state, selecting a new AA (including the gap character) uniformly from the 21 possibilities (ensuring it’s different from the current one). Each proposed mutation is accepted or rejected based on the change in total energy (ΔE) and the current, position-specific, β_j^i value, following the Metropolis criterion: accept if $\Delta E < 0$ or if a random number is less than $e^{-\beta_j^i \Delta E}$.
5. **Data Logging:** After the simulation is complete, it calls a dedicated function to write a detailed report file containing the initial parameters, the final sequence and energy for each β step, and overall acceptance statistics.

4.2 Simulated Annealing Schedule (β)

The β -values, which are inverse of the temperature ($1/k_B T$), are not constant; instead, they are defined through a mechanism, which we call the “engine”:

- **Engine:**

$$\beta_i^j = k_i \cdot \frac{\alpha}{S_j + \varepsilon}, \quad k_i = (c_r)^i \quad \forall i \in [1, N_\beta], j \in [1, L_{\text{chain}}]$$

where c_r is the cooling rate, ε avoids division by 0, α is a scale factor, and S_j is entropy for position j in human reference Schrödinger chain.

4.3 Energy Function

The total energy, E_{total} , is a weighted sum of three components, which we aim to minimize:

$$E_{\text{total}} = w_{\text{log}} E_{\text{log}} + w_{\text{prop}} E_{\text{prop}} + w_{\text{penalty}} E_{\text{penalty}}$$

The weights for each component (`WEIGHT_LOG`, `WEIGHT_PROP`, `WEIGHT_PENALTY`) are defined in ‘head.h’.

1. **Log Humanness Energy, E_{log} :** This term quantifies how “human-like” a sequence is based on AA frequencies from a reference set of human sequences (the Schrödinger chain):

$$E_{\text{log}} = - \sum_{i=1}^{L_{\text{chain}}} \log(p_i(a_i))$$

where $p_i(a_i)$ is the frequency of the amino acid a_i at position i in the human reference. A large penalty (`ZERO_FREQ_PENALTY_LOG`) is added for amino acids with a frequency below a small threshold (`EPSILON`).

2. **Property Distance Energy, E_{prop} :** This term measures the similarity between the properties of the sequence and the human reference; it's calculated as the squared Euclidean distance in the property space for each position:

$$E_{\text{prop}} = \sum_{i=1}^{L_{\text{chain}}} \sum_{j=1}^{N_{\text{props}}} \left(\text{prop}_j(a_i) - \overline{\text{prop}_j(i)} \right)^2$$

where $\text{prop}_j(a_i)$ is the value of property j for amino acid a_i , and $\overline{\text{prop}_j(i)}$ is the average value of property j at position i in the Schrödinger chain.

3. **Wandering Penatly Energy, E_{penalty} :** This term discourages the sequence from deviating too far from the original murine sequence; we thus only incentivize mutation if there's a relatively big gain in humanness. It consists of a penalty based on property distance and a (for now) constant penalty for any mutation:

$$E_{\text{penalty}} = \sum_{i=1}^{L_{\text{chain}}} \left[\left(\sum_{j=1}^{N_{\text{props}}} (\text{prop}_j(a_i) - \text{prop}_j(a_i^{\text{original}}))^2 \right) + P \cdot \delta_{a_i}^{\text{original}} \right]$$

where a_i^{original} is the amino acid at position i in the original murine sequence, P is the `AA_MUTATION_PENALTY`, and δ is the “generalized Kronecker delta” (logical if gate). This approach ensures each mutation has a minimum energy cost, while heavily penalizing big property changes.

4.4 Model Parameters

The following table summarizes the key parameters for metropolis in our simulations. These values are defined in `main.c` and `head.h`.

Parameter	Value (provisional)	Description
<code>n_betas</code>	50	Number of cooling steps in the annealing schedule.
<code>n_sweeps</code>	800	Number of Metropolis sweeps per β step.
<code>n_metropolis</code>	20	Number of independent runs per murine seed sequence.
<code>w_log</code>	0.4 (from <code>head.h</code>)	Weight for the log-humanness energy component.
<code>w_prop</code>	0.4 (from <code>head.h</code>)	Weight for the property-distance energy component.
<code>w_penalty</code>	0.2 (from <code>head.h</code>)	Weight for the wandering penalty energy component.
<code>cooling_rate</code> (c_r)	1.02	Multiplicative factor for increasing k_i at each step.
<code>scale_factor</code> (α)	1.0	Global scaling factor for all β values.
<code>AA_MUTATION_PENALTY</code> (P)	1.5 (from <code>head.h</code>)	Constant penalty added for any AA mutation away from the original.